

# Multi-Level Robotic Pouring Using Offline Reinforcement Learning

Xiaoxiong Zhang, Junwei Liu and Wei Zhang

**Abstract**—This paper presents a novel learning-based method for a robotic manipulator to achieve liquid pouring across different liquid levels using only visual sensors. Previous works have relied on either online reinforcement learning or imitation learning, which are limited by sim-to-real gaps or data efficiency. In this paper, we propose to combine supervised learning and offline reinforcement learning, utilizing a human demonstration dataset containing only successful pouring at the highest level. Specifically, our approach employs supervised learning for a visual classifier, transforming the visual input into a categorical distribution over liquid levels. The offline reinforcement learning method is applied to a binary-conditioned control policy, which takes a binary signal and the robot's proprioception as inputs to determine the action. The binary signal indicates whether the desired liquid level has been reached based on the target liquid level. Through experiments, our method demonstrates superior effectiveness compared to a state-of-the-art imitation learning algorithm. Moreover, tests in unseen scenarios and multiple liquid level commands verify the generalization and transferability of our approach.

## I. INTRODUCTION

The emergence of service robots in domestic, office, and hospitality settings has shown the need for these autonomous agents to perform tasks that are routine for humans but complex for robots. One such task is pouring liquids, which is simple for humans but poses a significant challenge for robots. This task encompasses intricate fluid dynamics, necessitates advanced perception capabilities, and requires precise motion control. The ability to pour liquids is not just a matter of convenience; it is an important skill that service robots must master to operate effectively in human environments.

Recent advancements in robotic capabilities have been significantly enhanced by the application of reinforcement learning (RL) methods. Studies such as [1], [2] have demonstrated the potential of training robotic arms in simulation environments to perform liquid pouring tasks using weight sensors as perceptual input. These approaches aim to model the dynamic process of liquid pouring, allowing for the prediction of liquid flow based on known parameters. However, they are constrained by the need for detailed

knowledge of liquid properties and the sim-to-real gap in policy deployment.

In response to these challenges, research has turned towards multimodal perception as a means to enhance the accuracy of liquid volume estimation. The authors in [3]–[6] have explored the use of acoustic and tactile sensors to provide a more nuanced understanding of the pouring process. Despite their promise, these approaches rely on sophisticated and costly sensor arrays, which are prone to interference from environmental noise. To overcome these limitations, [7] proposed a hierarchical imitation learning approach that allows robots to pour specified volumes of liquid using only visual sensors. While this method streamlines the perceptual process, it requires intricate design of control policies and detailed statistical analysis of demonstrations, making it cumbersome to implement.

In this paper, we present a novel learning method for a robotic arm to perform liquid pouring tasks according to specific liquid level commands. Instead of collecting successful human demonstrations for every liquid level, we utilize a demonstration dataset containing only the successful pourings with the highest liquid level, supplemented with liquid level labels. Our method mainly comprises a visual classifier and a binary-conditioned control policy. The visual classifier, trained via supervised learning on the demonstration dataset, transforms the visual input into a categorical distribution over the liquid levels. This distribution is then converted into a binary signal based on the liquid level command to indicate whether the desired liquid level has been reached. The binary-conditioned control policy takes the binary signal and the robot's proprioception as inputs to determine the action, which is trained using offline RL method with the dataset derived from the demonstration dataset.

Our contributions are two-fold. First, the proposed method is among the first that integrates supervised learning with offline RL to achieve liquid pouring tasks across different liquid levels. This approach accomplishes the task without requiring demonstrations for each specific liquid level, thereby enhancing the data efficiency in robotic manipulation. Second, a series of experiments demonstrate that our method surpasses a state-of-the-art end-to-end imitation learning approach in both performance and generalizability regarding the highest level demonstrations. Additionally, it showcases the ability to adapt to liquid level commands, achieving multi-level robotic pouring with remarkable precision.

This work was supported in part by the National Natural Science Foundation of China under Grant 62073159, and Grant 62303208; in part by the Shenzhen Key Laboratory of Control Theory and Intelligent Systems, under Grant ZDSYS20220330161800001; and in part by the Science, Technology, and Innovation Commission of Shenzhen Municipality under Grant JCYJ20220530115010023. X. Zhang, J. Liu and W. Zhang are with the Shenzhen Key Laboratory of Control Theory and Intelligent Systems, School of System Design and Intelligent Manufacturing, Southern University of Science and Technology, Shenzhen 518055, China 12433017@mail.sustech.edu.cn, {liujw, zhangw3}@sustech.edu.cn

Corresponding authors: Junwei Liu; Wei Zhang.

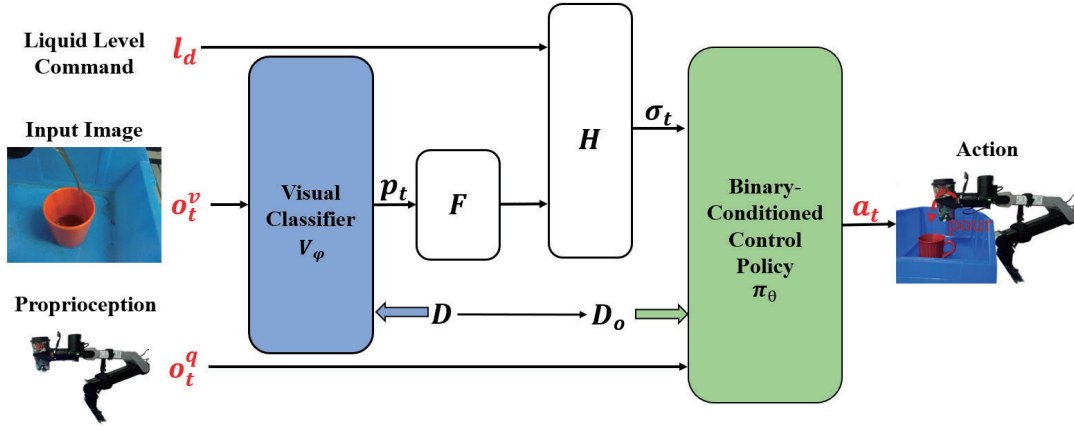


Fig. 1. The pipeline of our proposed method. The inputs of our algorithm include the liquid level command  $l_d$ , the visual information  $o_t^v$ , and the proprioception  $o_t^q$ , and the output is the action  $a_t$ . Specifically, the visual classifier  $V_\phi$ , trained on the demonstration dataset  $\mathbf{D}$  via supervised learning, processes  $o_t^v$  to produce an  $L$ -level softmax output  $p_t$ . This output is then transformed using  $F$ , and combined with the desired liquid level  $l_d$  to generate a binary signal  $\sigma_t$  through  $H$ . Finally, the binary signal  $\sigma_t$ , along with  $o_t^q$ , is fed into the binary-conditioned control policy  $\pi_\theta$  to infer the action  $a_t$ . The binary-conditioned control policy is trained on the offline RL dataset  $\mathbf{D}_o$ , derived from  $\mathbf{D}$ , using offline RL method.

## II. PROBLEM FORMULATION

This paper aims to learn from a demonstration dataset to enable a robotic arm to accurately pour liquids into various containers for different liquid level commands. We categorize each container with  $L$  liquid level degrees, labeled from 1 to  $L$ , where 1 represents the lowest and  $L$  represents the highest level. Unlike traditional imitation learning, our demonstration dataset contains only successful demonstrations of pouring liquids to the highest level, rather than providing demonstrations for all  $L$  different levels. To ensure self-adaptability to different liquid levels, we allow the liquid levels to be labeled within the demonstration dataset.

The demonstration dataset  $\mathbf{D} = \{\tau_i\}_{i=1}^N$  consists of  $N$  trajectories, where each trajectory  $\tau_i$  is defined as  $\{o_t, a_t, l_t\}_{t=0}^T$ . Here,  $o_t$  represents the observation at time  $t$ , including both the visual information of the environment  $o_t^v$  and the proprioceptive information of the robotic arm  $o_t^q$ ,  $a_t$  represents the action taken at time  $t$ , and  $l_t$  represents the liquid level at time  $t$ . Given a liquid level command  $l_d \in \{1, \dots, L\}$ , our goal is to learn a control policy from the demonstration dataset  $\mathbf{D}$ , which takes both the liquid level command  $l_d$  and the observation  $o_t$  as inputs and computes the corresponding action  $a_t$ , such that the robotic arm can automatically pour liquids to the  $l_d$ -th level.

## III. METHOD

### A. Overview

Our method comprises two key components: a visual classifier and a binary-conditioned control policy, organized into two parallel training steps. First, the visual classifier is trained using the demonstration dataset  $\mathbf{D}$  through supervised learning. This classifier takes the visual information  $o_t^v$  as input and outputs a categorical distribution over the liquid levels. Second, the binary-conditioned control policy is trained using offline RL method. The binary input signal indicates whether the desired liquid level has been reached,

and the offline RL dataset  $\mathbf{D}_o$  is derived from the demonstration dataset  $\mathbf{D}$ . During inference, two transformation logics are applied to integrate the visual classifier with the binary-conditioned control policy, ensuring seamless coordination between visual inputs and actions based on the liquid level command. The pipeline of our method is shown in Figure 1.

### B. Visual Classifier

1) *Visual Classifier*: The visual classifier, denoted as  $V_\phi$ , incorporates a visual encoder to leverage the powerful feature extraction capabilities of latest deep learning models. We select ResNet-18 [8] and initialize the parameters using a pre-trained model on ImageNet [9]. To represent a categorical distribution for  $L$  liquid levels, we replace the original classification head with a  $L$ -class liquid level classification head. As such, the visual classifier  $V_\phi$  transforms the visual input  $o_t^v$  into a  $L$ -level softmax output, denoted as  $p_t$ .

2) *Data Augmentation*: To enhance the robustness of the visual encoder in handling diverse camera angles, lighting conditions, and other noise factors present in real-world scenarios, we employ three image data augmentation techniques to diversify the image data: (1) applying horizontal flips to the images with a probability of 50%, (2) randomly rotating the images by  $\pm 15^\circ$  to improve the model's robustness to rotation, and (3) randomly adjusting the brightness, saturation, contrast, and hue of the images to simulate potential lighting variations. These techniques help reduce overfitting and enable the visual encoder to learn the essential features of the classification task more effectively.

3) *Loss Function*: The visual classifier is trained using the visual input  $o_t^v$  and the liquid level  $l_t$  from the demonstration set  $\mathbf{D}$ . During implementation, we observed that samples belonging to the lowest and highest levels were more prevalent compared to samples of other levels. To address this level imbalance issue, we employ a weighted cross-entropy loss

function for training the visual classifier:

$$L = - \sum_{k=1}^L w_k \left( \sum_i \mathbf{1}_{\{l_i=k\}} \log(\mathbf{V}_\varphi(o_i^v)) \right)$$

where  $\mathbf{1}_{\{l_i=k\}}$  is the indicator function that equals 1 if  $l_i = k$  and 0 otherwise, and  $w_k$  are the weights assigned to each level to balance the loss contributions.

### C. Binary-Conditioned Control Policy

1) *Offline RL Dataset*: Offline reinforcement learning (RL) is a branch of RL that trains policies using offline datasets, eliminating the need for interaction with the environment during training [10]. Similar to traditional RL [11], offline RL problems are modeled as a Markov decision process  $M = \{S, A, P, R, \gamma, d_0\}$ , where  $S$  represents the state space,  $A$  represents the action space,  $P(s'|s, a)$  defines the state transition probability,  $R(s, a)$  represents the reward function,  $\gamma$  is a scalar discount factor, and  $d_0$  defines the initial state distribution.

The offline RL dataset, denoted as  $\mathbf{D}_o$ , commonly takes the form  $\{(s_t, a_t, r_t, s_{t+1})\}$ . Here,  $s_t$  represents the state at time  $t$ ,  $a_t$  represents the action taken in the state  $s_t$ ,  $r_t$  represents the immediate reward obtained after taking the action  $a_t$ , and  $s_{t+1}$  represents the next state after taking the action. We first define the state  $s_t$  in the context of multi-level robotic pouring. Typically, the state should include the status of the environment and the robotic manipulator, represented by the visual information  $o_t^v$  and the proprioception  $o_t^q$ . To better decouple the visual information from the visual classifier and to achieve the multi-level pouring task, we replace the visual information  $o_t^v$  with a binary signal  $\mathbf{H}(l_t, l_{ex})$  based on the liquid level  $l_t$ , as follows:

$$\mathbf{H}(l_t, l_{ex}) = \begin{cases} 1 & \text{if } l_t \geq l_{ex} \\ 0 & \text{if } l_t < l_{ex} \end{cases}$$

where  $l_{ex}$  is the expected liquid level. Since all demonstrations in the dataset  $\mathbf{D}$  involve filling the container to the highest level, the expected liquid level  $l_{ex}$  is specified as  $L$  during training (while during inference,  $l_{ex}$  is specified as liquid level command  $l_d$ ). This binary signal simplifies the state representation by focusing on whether the desired liquid level has been reached.

Another crucial element of the offline RL dataset is the reward function, which reflects the quality of the action based on the current state. For liquid pouring, the reward function should be designed to reflect whether the liquid is within the desired level. With the liquid level  $l_t$ , the reward function is given as follows:

$$r_t = \begin{cases} -1 & \text{if pour, } l_t \geq l_{ex} \\ 0 & \text{if not pour} \\ 1 & \text{if pour, } l_t < l_{ex}. \end{cases}$$

This reward function ensures that the robotic arm is penalized  $r = -1$  for pouring when the liquid level has already reached or exceeded the desired level, as this would cause overfilling. Conversely, the robotic arm is rewarded  $r = 1$  for pouring

when the liquid level is below the target, encouraging it to reach the desired level. The robotic arm receives no reward or penalty  $r = 0$  for not pouring to maintain a neutral stance.

Having the state representation  $s_t = (\mathbf{H}(l_t, L), o_t^q)$  and the defined reward function  $r_t$ , we can construct an offline RL dataset  $\mathbf{D}_o = \{(s_t, a_t, r_t, s_{t+1})\}$  from the demonstration dataset  $\mathbf{D}$ . This offline RL dataset will then be used to train a control policy.

2) *Model Training*: The goal of offline RL is to learn a stochastic policy  $\pi_\theta(a_t|s_t)$  parameterized by  $\theta$  that maximizes the expected cumulative discounted return  $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$  based on the offline RL dataset  $\mathbf{D}_o$ . The control policy  $\pi_\theta$  can be derived by maximizing the action-value function  $Q^{\pi_\theta}$ :

$$\pi_\theta = \arg \max_{\theta} \mathbb{E}_{s_t \sim \mathbf{D}_o, a_t \sim \pi_\theta(a_t|s_t)} [Q^{\pi_\theta}(s_t, a_t)]. \quad (1)$$

However, one of the challenges in offline RL is that the state distribution in (1) confines to the offline RL dataset, but differs from the distribution encountered during policy inference. This discrepancy, known as distribution shift [10], [12]–[15], leads to deviations in the estimation of the action-value function and subsequently results in suboptimal policy performance. Addressing this issue is crucial for ensuring the trained policy can generalize effectively to real-world scenarios and perform well when deployed.

To address this challenge, we propose to maximize objective function (2) that is similar to TD3+BC [16], which mitigates distribution shift by narrowing the gap between actions output by policy and actions in the dataset  $\mathbf{D}_o$ :

$$\mathbb{E}_{(s_t, a_t) \sim \mathbf{D}_o} [\alpha Q^{\pi_\theta}(s_t, \pi_\theta(s_t)) - (\pi_\theta(s_t) - a_t)^2] \quad (2)$$

where  $\pi_\theta(s_t)$  is a deterministic policy that outputs a specific action rather than a distribution of possible actions, and the weight  $\alpha$  aims to adjust the balance between the two loss terms in the objective function. As the scale of the Q-value function can vary across different offline datasets,  $\alpha$  is normalized as  $\alpha = \frac{\eta}{\mathbb{E}_{(s_t, a_t) \sim \mathbf{D}_o} [Q^{\pi_\theta}(s_t, a_t)]}$ , where  $\eta$  is a constant hyperparameter.

To maximize the objective function (2), we approximate the action value function  $Q^{\pi_\theta}$  using a critic network. The critic network is trained by incorporating the double Q-networks technique [17] to enhance the stability of the training process. Specifically, we initialize two Q-networks  $Q_{\phi_1}$  and  $Q_{\phi_2}$ , along with their corresponding target networks  $Q_{\phi'_1}$  and  $Q_{\phi'_2}$ , as well as the target policy network  $\pi_{\theta'}$ . The parameters  $\phi_i$  for  $i = \{1, 2\}$  are optimized by minimizing the following objective function:

$$\mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathbf{D}_o} [\|\hat{Q}(s_t, a_t) - Q_{\phi_i}(s_t, a_t)\|^2]. \quad (3)$$

The optimization objective is to minimize the temporal difference (TD) error of the action value function, where  $\hat{Q}(s_t, a_t)$  denotes the TD target of the action value function, computed as follows:

$$\hat{Q}(s_t, a_t) = r(s_t, a_t) + \gamma \min_{i=1,2} Q_{\phi'_i}(s_{t+1}, \pi_{\theta'}(s_{t+1})). \quad (4)$$

---

**Algorithm 1** Control Policy Training Process

---

Initialize  $\pi_\theta, Q_{\phi_1}, Q_{\phi_2}$ ; Target network  $\pi_{\theta'}, Q_{\phi'_1}, Q_{\phi'_2}$   
**for** each iteration **do**  
    Sample transition batch  $\mathbf{B} = \{(s_t, a_t, r_t, s_{t+1})\} \sim \mathbf{D}_o$ .  
    **#Critic Optimization**  
    Update  $Q_{\phi_1}$  and  $Q_{\phi_2}$  by minimize Equation (3) and (4)  
    **#Actor Optimization**  
    Update policy  $\pi_\theta$  by maximize Equation (2)  
    **#Update Target networks**  
     $\theta' = (1 - \rho)\theta + \rho\theta$ ;  $\phi'_i = (1 - \rho)\phi'_i + \rho\phi_i$   
**end for**

---

Our proposed binary-conditioned control policy training process is summarized in Algorithm 1.

#### D. Inference

For inference, we need to integrate the visual classifier with the binary-conditioned control policy. The output of the visual classifier, an  $L$ -level softmax  $p_t$ , must be transformed into a binary signal. This binary signal will indicate whether the current liquid height is within the desired liquid level. To mitigate frequent changes in the predicted classes at critical states, we adopt the following rule: the model outputs the predicted level only when its probability exceeds a predefined threshold; otherwise, the output is maintained as the previous prediction. This transformation logic is defined as follows:

$$\mathbf{F}(p_t) = \begin{cases} c'_{t-1} & \text{if } \max(p_t) < \rho \\ c_t & \text{if } \max(p_t) \geq \rho \end{cases}$$

where  $c_t \in \arg\max\{p_t\}$  represents the level index corresponding to the output of the visual classifier at time step  $t$ ,  $c'_{t-1}$  denotes the outcome of the level index at the previous time step  $t-1$ , and  $\rho$  is a hyperparameter representing the probability threshold.

The inference process is detailed as follows. At time step  $t$ , the robotic arm receives the observation  $o_t = (o_t^v, o_t^q)$ , where  $o_t^v$  is the visual input and  $o_t^q$  is the robot's proprioception. The visual classifier first estimates the liquid level  $p_t = \mathbf{V}_\phi(o_t^v)$ . This estimate is then transformed to obtain the binary signal  $\sigma_t = \mathbf{H}(\mathbf{F}(p_t), l_d)$ , where  $l_d$  is the liquid level command. The state of the offline RL is derived by combining  $\sigma_t$  and  $o_t^q$ , i.e.,  $s_t = (\sigma_t, o_t^q)$ , which is fed to the binary-conditioned control policy  $\pi_\theta(s_t)$  to yield the action command  $a_t$  for the robot to execute. The process repeats with the next observation  $o_{t+1}$ .

## IV. EXPERIMENT

The efficacy and generalizability of the proposed algorithm were evaluated through two sets of experiments. The first set aimed to assess the efficacy of the method by having the algorithm perform the same task as observed in the dataset, i.e., filling the container to the  $L$  liquid level. The second set was designed to evaluate the method's transferability by instructing it to fill the container to other target levels. The

details of the experimental procedures and the corresponding results are elaborated upon in the subsequent sections.

The key objectives of these experiments are as follows: 1) To validate the effectiveness of the proposed methodology; 2) To assess the transferability of the method by evaluating its performance on liquid levels outside the range observed in the dataset; 3) To examine the generalizability of the method by evaluating its performance in scenarios not encountered in the training dataset.

#### A. Experiment Setup

In the experiment, we set  $L = 5$ , representing a total of 5 liquid levels, namely 0%-20%, 20%-40%, ..., 80%-100%. The labeling of these liquid levels is manageable since the liquid level changes continuously within a trajectory  $\tau = \{o_t, a_t\}$ . Hence, we only need to identify several key frames (e.g., 20%, 40%) to complete the liquid level labeling for the entire trajectory. Additionally, we observed that the number of samples corresponding to each liquid level category is imbalanced. To tackle this problem, we assigned lower weights of  $w_1 = 3$  and  $w_5 = 1$  to the loss terms related to level 1 and 5 respectively, while applying higher weights of  $w_2 = w_3 = w_4 = 5$  to the remaining level samples.

To verify our method, experiments were conducted in the scenario presented in Figure 2. The visual information  $o_t^v$ , which is a  $640 \times 480$  resolution RGB image of the liquid in the container captured by a side-view camera (RealSense D435f), along with the robot's proprioception  $o_t^q$ , is transmitted to the computer through the ROS system. After obtaining this information, the computer inputs it into the algorithm to calculate the action  $a_t$  to be taken, that is, the joint position the robot should reach in the next moment. Subsequently, the action is sent to the robotic arm via the ROS system for execution. The ARX5 robotic arm is utilized in this process.

We collected demonstrations of pouring to the 80% - 100% level in 12 different scenarios using the teleoperation method of ALOHA [18], with two demonstrations collected for each scenario, altogether constituting a dataset  $D = \{\tau_i\}_{i=1}^{24}$  of 24 demonstrations. In the subsequent experiments, we added some new scenarios that did not appear in the dataset to test the generalization of the algorithm. All the scenarios are summarized in Table III.

#### B. Effectiveness

In this group of experiments, the aim is to verify the effectiveness of the proposed method. Specifically, we perform the same task as in the dataset, that is, pouring the liquid to a liquid level of 80%-100% and comparing it with the current state-of-the-art imitation learning algorithm ACT [18]. We conducted 50 experiments in the seen scenarios and 30 experiments in the unseen scenarios. The experimental results are summarized in Table I.

The hyperparameters of the ACT algorithm follow the default settings in the paper [18]. It can be found that in the seen and unseen scenarios, the success rates of our proposed method are 98% and 93.3% respectively, while those of ACT



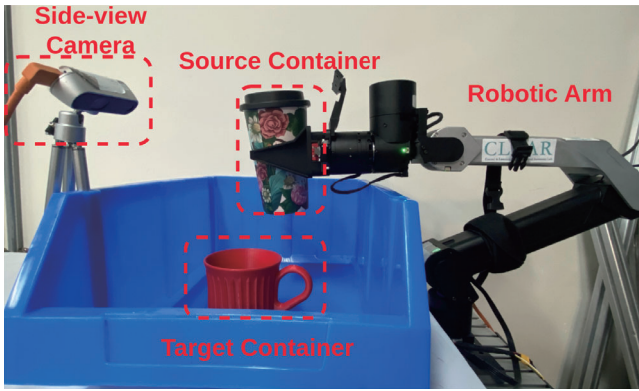


Fig. 2. The experimental setup: During data collection, the robotic arm is controlled by a human operator through teleoperation. While during deployment, the robotic arm is controlled by our proposed algorithm to achieve multi-level pouring tasks automatically.

TABLE I  
EXPERIMENT RESULT

| Scenario       | Success/Trials |       | Success Rate |               |
|----------------|----------------|-------|--------------|---------------|
|                | ACT            | Ours  | ACT          | Ours          |
| 1-12 (seen)    | 30/50          | 49/50 | 60%          | <b>98%</b>    |
| 12-20 (unseen) | 12/30          | 28/30 | 40%          | <b>93.33%</b> |
| total          | 42/80          | 77/80 | 52.5%        | <b>96.25%</b> |

are only 60% and 40% respectively. Obviously, our method is significantly superior to the ACT algorithm in both cases, effectively demonstrating the effectiveness of our proposed method.

The overall success rate of 96.25% achieved in this experimental evaluation demonstrates the efficacy of the proposed method. Furthermore, the 93.33% success rate observed in the scenarios not represented in the training dataset substantiates the strong generalization capabilities of the developed approach, underscoring its ability to adapt to diverse experimental conditions.

To elucidate the efficacy of the probabilistic threshold  $F(\cdot)$ , we set the hyperparameter probability threshold  $\rho$  to 75% during the experimental trials. Specifically, we conducted experiments using Scenario 14, and the results are depicted in Figure 3. Based on the observation that the output of the visual encoder becomes highly unstable as the liquid level is about to undergo a transition, this unstable output can lead to frequent changes in the state passed to the control policy, resulting in erratic actions. By applying the probability threshold strategy, the rapid fluctuations in the visual encoder's output are filtered out, making the changes more gradual. While this may introduce a slight time delay in the output, it does not significantly impact the control policy.

### C. Transferability

The primary objective of this series of experiments was to evaluate the transferability of the proposed method. The control policy was trained on a dataset where the container

TABLE II  
LIST OF TARGET CONTAINERS USED

| ID          | 1                                                                                   | 2                                                                                   | 3                                                                                   | 4                                                                                   | 5                                                                                   |
|-------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Container   |  |  |  |  |  |
| Volume (ml) | 206                                                                                 | 248                                                                                 | 144                                                                                 | 216                                                                                 | 165                                                                                 |

TABLE III  
EXPERIMENT SCENARIO

| Scenario |    | Container | Liquid     |
|----------|----|-----------|------------|
| Seen     | 1  | 2         | Black Tea  |
|          | 2  | 2         | Green Tea  |
|          | 3  | 2         | Oolong tea |
|          | 4  | 3         | Black Tea  |
|          | 5  | 3         | Green Tea  |
|          | 6  | 3         | Oolong tea |
|          | 7  | 4         | Black Tea  |
|          | 8  | 4         | Green Tea  |
|          | 9  | 4         | Oolong tea |
|          | 10 | 5         | Black tea  |
|          | 11 | 5         | Green tea  |
|          | 12 | 5         | Oolong tea |
| Unseen   | 13 | 1         | Black Tea  |
|          | 14 | 1         | Green Tea  |
|          | 15 | 1         | Oolong tea |
|          | 16 | 1         | Juice      |
|          | 17 | 2         | Juice      |
|          | 18 | 3         | Juice      |
|          | 19 | 4         | Juice      |
|          | 20 | 5         | Juice      |

was filled to 80%-100% capacity. We sought to investigate whether this trained policy could also successfully fill the container to other specified levels.

Specifically, we assigned varied  $l_d$  values (e.g., 1, 2, etc.) and tested the algorithm's transferability across a total of six scenarios, with one scenario represented in the training dataset, and the remaining five being novel, unseen situations. For each scenario, we conducted five experimental trials, resulting in a total of 30 trials. The algorithm successfully completed 29 of these trials, yielding an overall success rate of 96.67%. The results of this evaluation are summarized in Table IV.

The high success rate achieved by our method in completing the multi-level liquid pouring tasks across a range of specified  $l_d$  values demonstrates the transferability of our approach. Furthermore, the consistent performance of our method across a variety of scenarios further underscores its robust generalization capabilities.

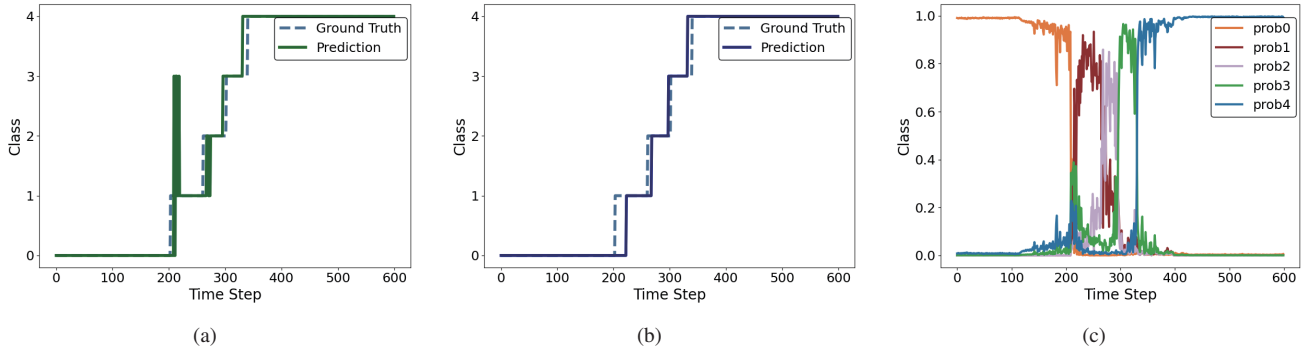


Fig. 3. This figure takes a trajectory in Scenario 14 as an example to illustrate the role of the probability threshold strategy. Figure (a) represents the raw output of the visual encoder, and it can be found that its output changes frequently when the class is about to transfer, which is very unfavorable for the subsequent control of the control policy. Figure (b) shows the output class after the probability threshold strategy, and it can be found that it eliminates the frequent changes in the critical situation. Figure (c) shows the probabilities of the five classes throughout the process. It can be found that in the critical state, the visual encoder is not confident enough to determine the current class, so there are similar probabilities of multiple classes, resulting in frequent changes in the output class.

TABLE IV  
EXPERIMENT RESULT

| Scenario     | $l_d$ | Success/Trials |
|--------------|-------|----------------|
| 12 (seen)    | 2     | 5/5            |
| 15 (unseen)  | 3     | 5/5            |
| 17 (unseen)  | 4     | 5/5            |
| 18 (unseen)  | 4     | 4/5            |
| 19 (unseen)  | 3     | 5/5            |
| 19 (unseen)  | 4     | 5/5            |
| Success Rate |       | 96.67%         |

## V. CONCLUSION

In this paper, we propose a vision sensor-dependent method for multi-level robotic pouring based on supervised learning and offline reinforcement learning. Through a series of experiments, we have demonstrated the effectiveness, transferability, and generalization of the proposed method. In the effectiveness experiment, our method has a success rate about 40% higher than the state-of-the-art imitation learning algorithm ACT, which fully indicates the effectiveness of our method. In the transferability experiment, for other liquid levels, our proposed method has a success rate of 96.67%, which fully proves the transferability of our method and its ability to complete the multi-level robotic pouring task. Moreover, in both of the above experiments, we have included scenarios that were not seen in the dataset. In these scenarios, our method still has a high success rate, which fully demonstrates the generalization of our method.

## REFERENCES

- [1] C. Do, C. Girdillo, and W. Burgard, "Learning to pour using deep deterministic policy gradients," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3074–3079, IEEE, 2018.
- [2] E. Babaian, T. Sharma, M. Karimi, S. Sharifzadeh, and E. Steinbach, "Pournet: Robust robotic pouring through curriculum and curiosity-based reinforcement learning," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 9332–9339, IEEE, 2022.
- [3] H. Liang, C. Zhou, S. Li, X. Ma, N. Hendrich, T. Gerkmann, F. Sun, M. Stoffel, and J. Zhang, "Robust robotic pouring using audition and haptics," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 10880–10887, IEEE, 2020.
- [4] A. Burns, S. Xiang, D. Lee, L. Jackel, S. Song, and V. Isler, "Look and listen: A multi-sensory pouring network and dataset for granular media from human demonstrations," in *2022 International Conference on Robotics and Automation (ICRA)*, pp. 2519–2524, IEEE, 2022.
- [5] H. Liang, S. Li, X. Ma, N. Hendrich, T. Gerkmann, F. Sun, and J. Zhang, "Making sense of audio vibration for liquid height estimation in robotic pouring," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5333–5339, IEEE, 2019.
- [6] P. Piacenza, D. Lee, and V. Isler, "Pouring by feel: An analysis of tactile and proprioceptive sensing for accurate pouring," in *2022 International Conference on Robotics and Automation (ICRA)*, pp. 10248–10254, IEEE, 2022.
- [7] D. Zhang, Q. Li, Y. Zheng, L. Wei, D. Zhang, and Z. Zhang, "Explainable hierarchical imitation learning for robotic drink pouring," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 4, pp. 3871–3887, 2021.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- [9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009.
- [10] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," *arXiv preprint arXiv:2005.01643*, 2020.
- [11] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [12] S. Fujimoto, D. Meger, and D. Precup, "Off-policy deep reinforcement learning without exploration," in *International conference on machine learning*, pp. 2052–2062, PMLR, 2019.
- [13] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1179–1191, 2020.
- [14] Z. Wang, J. J. Hunt, and M. Zhou, "Diffusion policies as an expressive policy class for offline reinforcement learning," in *The Eleventh International Conference on Learning Representations*.
- [15] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit q-learning," in *Deep RL Workshop NeurIPS 2021*.
- [16] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," *Advances in neural information processing systems*, vol. 34, pp. 20132–20145, 2021.
- [17] H. Hasselt, "Double q-learning," *Advances in neural information processing systems*, vol. 23, 2010.
- [18] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv preprint arXiv:2304.13705*, 2023.